

自适应门控事实记忆：从选择性长期存储到精确词元恢复

作者姓名

单位名称
电子邮箱

1 问题背景

从全局注意力到固定状态推理

标准自注意力允许第 t 个 token 访问此前全部位置。对于长度为 L 、隐藏维度为 D 的序列，其训练计算量随 L^2 增长；自回归推理虽然可以复用历史键值，仍需保存与历史长度成正比的 KV cache (Vaswani et al. 2017)。当对话持续增长时，每个新 token 都会带来新的状态，显存占用和单步读取成本也随之增加。固定状态模型试图消除这种无界增长，但只要历史信息被压缩或覆盖，就必须回答一个根本问题：在有限状态中应保留什么，才能避免真正有用的信息被无关内容淹没。

基于滑动窗口的因果语言模型用固定长度的键值缓存控制推理状态。设局部注意力预算为 B ，在时间步 t ，模型只能访问少量初始注意力汇聚词元与最近的上下文：

$$C_t = C_{\text{sink}} \cup C_{\text{recent}}, \quad |C_t| \leq B. \quad (1)$$

在每层只读取 B 个局部键值时，注意力计算量约为 $\mathcal{O}(LBD)$ ，流式 KV 状态约为 $\mathcal{O}(N_{\text{layer}}BD)$ ，二者在 B 固定时不再随完整历史无限增加。保留少量 sink token 可以缓解窗口滑过序列开头后出现的注意力退化 (Xiao et al. 2024)。近期大规模比较进一步指出，带 sink 的简单 SWA 在多个短上下文和长上下文基准上可达到或超过复杂的后训练线性注意力方法，同时具有很强的速度和内存竞争力 (Jolicoeur-Martineau et al. 2026)。这一观察提示我们：评价新型记忆机制时，必须把结构简单、实现成熟的 SWA-sink 作为强基线，并分别报告质量、状态和计算成本。

不过，SWA 的固定成本来自主动丢弃窗口外的逐 token 表示。它能利用多层感受野与模型参数中的统计知识，却不能保证任意早期事实仍可在数千 token 后被逐字恢复。对长期对话而言，真正需要保存的内容通常只占历史的一小部分。例如，用户明确要求长期保存的偏好可能需要跨越数千个干扰词元，而临时说明、无关故事和一次性事实则不应持续占用状态。因此，长期记忆的关键不只是扩大窗口，还包括判断什么内容值得写入、已存内容是否应继续保留，以及查询时应读取哪一条记忆。

选择性记忆问题的形式化

设一段对话由轮次 $x_1, \dots, x_T$ 组成，每轮可能产生事实候选集合 $\mathcal{F}_t$ 。系统维护最多 M 个长期槽位 $\mathcal{M}_t$ ，并满足

$$|\mathcal{M}_t| \leq M, \quad \mathcal{M}_{t+1} = U(\mathcal{M}_t, \mathcal{F}_t), \quad (2)$$

其中 U 同时包含拒绝、写入、合并、更新和淘汰。对后续查询 q ，reader 从 $\mathcal{M}_t$ 中选出少量槽位 $R(q, \mathcal{M}_t)$ ，语言模型再根据局部上下文与检索结果生成答案 y 。由此，端到端正确率至少受四类事件共同影响：目标事实是否被写入，是否在干扰后仍然存活，是否被 reader 选中，以及检索表示是否能被解码为正确 token 序列。

这种分解使不同失败模式可以被区分。若 survival 下降，问题在容量分配或保留；若 survival 高而 Recall 低，问题在语义寻址；若二者均高而 EM 低，问题位于融合或答案解码。本文的架构和实验均围绕这一分解组织，避免用单个端到端分数掩盖内部机制的差异。

从语义记忆到精确词元的第二个瓶颈

固定槽位记忆能够限制状态规模，但简单的“见到事实即写入”策略会产生容量竞争。当临时事实不断进入有限槽位时，早期的重要事实可能被最近最少使用 (LRU) 规则淘汰。内容门控可缓解这一问题：模型学习写入概率和保留概率，只让重要事实进入长期状态。然而，长期事实问答还包含另一个容易被忽略的环节，即从检索到的连续记忆表示恢复精确答案词元。一个槽位可以在语义上表示“Bob 的偏好与蓝色有关”，却未必能使语言模型在多个颜色词中稳定地把 blue 排在第一位。

本研究将长期事实恢复分解为三个相互关联但可分别测量的阶段：选择性存储、语义检索和词级生成。实验关注两个问题：其一，内容门控的有界槽位能否在长延迟和事实噪声下持续保存目标事实；其二，当目标槽位已经被正确检索时，显式的值片段建模能否缩小连续语义表示与离散答案词元之间的差距。进一步地，我们考察稀疏混合专家 (Mixture of Experts, MoE) 是否与该记忆路径稳定组合，以及增加前馈网络容量能否替代词级对齐机制。

2 架构设计与机制解读

双时间尺度的因果状态

模型以六层 decoder-only TinyLM 为骨干，同时维护局部词元缓存和长期事实槽位。局部通道保留 4 个 sink token 与 124 个 recent KV，承担精确的短期依赖；长期通道包含 $M = 8$ 个用户事实槽位，以压缩表示保存跨轮信息。两类状态共同受固定预算约束，因此在持续流式推理中不会随历史长度线性增长。

在时刻 t ，完整对话状态记为

$$\mathcal{S}_t = \{\mathcal{K}_t^{1:N}, \mathcal{V}_t^{1:N}, \mathcal{M}_t^{\text{user}}, \mathcal{M}_t^{\text{asst}}, \mathcal{P}_t, \mathcal{R}_t, \tau_t\}. \quad (3)$$

其中 $\mathcal{K}_t^{1:N}, \mathcal{V}_t^{1:N}$ 是每层 sink/recent KV; $\mathcal{M}_t^{\text{user}}$ 是用户事实槽位; $\mathcal{M}_t^{\text{asst}}$ 是四个固定角色的助手摘要槽位; $\mathcal{P}_t$ 是尚未提交的候选缓冲; $\mathcal{R}_t$ 缓存当前轮选中的检索索引; τ_t 记录绝对位置和轮次边界。所有张量都支持 batch 重排和按行 reset, 使每条会话能够在批处理过程中独立移动。

若忽略小型元数据, 完整状态的规模约为

$$\mathcal{O}(NBD) + \mathcal{O}(MD + MP), \quad (4)$$

其中 N 是层数, B 是局部 KV 预算, M 是事实槽位数, P 是每槽位保存的最大 payload 长度。本文固定 $N = 6, B = 128, M = 8, P = 12$, 所以生成历史继续增长时, 状态上界保持不变。这里的固定指张量规模上界固定, 并不意味着所有槽位都对每个 token 参与计算: 事实读取只在轮次边界发生, 融合层只接收 Top- K 结果。

模型以“用户输入及其后完整助手回复”为一轮。新一轮查询开始时, reader 只执行一次事实检索, 选中的槽位在本轮生成期间保持不变。当前轮的输出 logits 计算完成后, 候选事实才被提交到状态, 因此新写入的信息只能影响后续轮次。这种提交顺序保证了因果性, 也使整轮训练与分块流式推理具有一致的状态语义。

具体生命周期包含五个步骤。首先, 用户或系统输入打开新一轮, 并在最后一个 prompt token 处完成检索; 该位置的 logits 用于预测第一个助手 token, 因此长期记忆可以影响回答开头。其次, 用户 token 的候选 span 被写入有界 pending buffer, 但此时不改变可读槽位。第三, 助手生成过程中复用同一组检索结果, 避免 token 间路由抖动, 并以运行和与计数器累计本轮助手摘要。第四, 收到 commit 信号时, 模型先完成当前 token 的 logits, 再更新事实槽位和助手槽位。最后, 轮次关闭, 下一轮才可以读取新提交的事实。测试中整轮调用与“用户 prefill 加多个助手 chunk”的状态和 logits 一致, 验证了这套生命周期的流式语义。

事实抽取与语义槽位

对用户轮中的候选事实片段 S_i , 抽取器首先对片段内隐藏状态做聚合, 并生成语义键和值:

$$\bar{h}_i = \frac{1}{|S_i|} \sum_{j \in S_i} h_j, \quad k_i = W_k \bar{h}_i, \quad v_i = W_v \bar{h}_i. \quad (5)$$

键 k_i 用于内容寻址, 值 v_i 保存关系和事实整体语义。槽位还保存长度受限的 payload 词元、年龄、访问次数、置信度和来源等元数据, 便于审计读写行为并为保留控制器提供依据。每个事实最多保存 12 个 payload 词元, 状态上限由槽位数和 payload 上限共同确定。

候选起点由 user-role token 上的 start head 评分, length head 决定从起点向后复制的 token 数。候选数量和 payload 长度都预先设定上限, 因此抽取器不会把整段用户消息无界复制到长期状态。训练数据提供事实起点和长度标签; 临时事实仍具有与目标相同的“人物-关系-颜色”句式, 只是其重要性标签不同。这一构造迫使写入门在形式相近的候选之间作选择。

写入、合并与保留门控

写入控制器依据候选表示及上下文预测写入概率

$$g_i^w = \sigma(f_w(\bar{h}_i, m_t)), \quad (6)$$

其中 m_t 表示当前记忆状态的汇总信息。训练阶段采用带直通估计的软阈值, 使离散写入决定仍可接受监督; 评估阶段按阈值执行确定性写入。通过写入门的候选首先与已有键比较: 若最高相似度超过合并阈值, 则由更新门插值新旧表示; 否则写入空槽位。当没有空槽位时, 控制器根据保留分数选择淘汰对象。

对于被寻址到的槽位 r , 更新门根据旧 value、候选 value 与键相似度产生 $g_{i,r}^{\text{upd}}$, 并执行

$$v'_r = (1 - g_{i,r}^{\text{upd}})v_r + g_{i,r}^{\text{upd}}v_i. \quad (7)$$

键和 lexical value 使用相同的门控插值, 离散 payload 则在硬写入发生时替换。若候选与现有事实不够相似且槽位已满, Gated 条件淘汰保留分数最低的槽位; Fixed-LRU 条件使用最后访问时间最早的槽位, 并把写入概率与保留概率固定为 1, 从而形成不依赖随机控制头的确定性基线。

对已激活槽位 r , 保留控制器预测

$$g_r^{\text{ret}} = \sigma(f_{\text{ret}}(k_r, v_r, \text{meta}_r)), \quad (8)$$

低于阈值的槽位在下次提交边界被释放。写入门回答“新事实是否值得进入长期状态”, 保留门回答“旧事实是否仍值得占用槽位”; 二者共同把记忆容量分配转化为内容相关决策, 而不再仅由到达时间决定。

保留特征由槽位 key、value 以及 active strength、confidence、age、access count 和 conflict flag 组成。age 和访问次数经过缩放后输入线性 head。这里的 conflict 只表示新旧 payload 在重叠位置出现差异, 是一种保守的冲突诊断, 并不是完整的实体属性矛盾解析器。因此, 当前架构具备合并与覆盖接口, 但实验并未声称已经解决开放域知识更新。

稀疏读取与门控融合

查询表示 q_t 与所有活跃事实键计算语义相似度, 并只读取得分最高的 $K = 4$ 个槽位:

$$\mathcal{I}_t = \text{TopK}_r(q_t^\top k_r), \quad \tilde{m}_t = \sum_{r \in \mathcal{I}_t} \alpha_r v_r, \quad (9)$$

其中 α_r 是在候选集合内归一化的注意力权重。检索结果通过共享记忆融合模块注入第 3 和第 6 个解码层。对层 ℓ , 融合可写为

$$h_t^{(\ell)} \leftarrow h_t^{(\ell)} + g_t^{(\ell)} F_\ell(h_t^{(\ell)}, \tilde{m}_t), \quad (10)$$

其中 $g_t^{(\ell)}$ 控制长期记忆对当前 token 表示的影响。该设计将“选中哪条事实”和“注入多少记忆信号”分开, 使读路由与生成融合可以独立诊断。

reader 的 query 来自新一轮 prompt 边界, 事实键在写入时已归一化。读取得分还结合槽位 active strength, 使失活槽位无法参与排序。Top- K 后, 访问时间和访问次数被更新, 为后续保留和 LRU 审计提供状态。由于本实验最多只有一条真正需要长期保存的事实, Recall@1 是最直接的语义寻址指标; Recall@4 则检查目标是否至少进入实际送往融合层的候选集合。

值片段与词元对齐路径

旧架构以完整事实的平均表示作为语义 value, 并把完整 payload 的词嵌入再次平均后参与融合。这种表示适

Table 1: 双时间尺度状态中各组件的作用与上界。

组件	保存内容	实验上界
局部 KV	精确短期 token 表示	每层 4 + 124
用户事实槽位	跨轮语义事实和价值	8 槽
助手槽位	固定角色摘要	4 槽
事实 payload	可审计源 token	12/槽
轮内检索缓存	Top- K 槽位索引	$K = 4$
pending buffer	尚未提交的候选	1/轮

合检索，却弱化了答案对象的边界、顺序和离散身份。改进后的模型在保留原语义路径的同时，为事实中的目标值 $V_i \subset S_i$ 增加起点和长度监督：

$$\hat{s}_i = \text{softmax}(W_s H_{S_i}), \quad \hat{l}_i = \text{softmax}(W_l H_{S_i}). \quad (11)$$

模型只聚合值片段内部的隐藏状态，得到独立的词汇值表示

$$z_i = W_{\text{lex}} \left(\frac{1}{|V_i|} \sum_{j \in V_i} h_j \right). \quad (12)$$

槽位新增 lexical value、值词元 ID 和有效位置 mask；融合输入由原语义 value 扩展为 $v_i + z_i$ 。完整事实表示继续承担关系理解和语义检索，值表示则保留精确回答所需的局部词汇信息。

为使 z_i 可直接解码为单词或多词元值，我们为答案位置 p 加入可学习位置嵌入，并使用与语言模型词嵌入共享的词表投影：

$$o_{i,p} = E \text{ LN}(W_o(z_i + e_p)), \quad \mathcal{L}_{\text{tok}} = - \sum_p \log P(y_{i,p} | z_i, p). \quad (13)$$

该损失同时施加在写入轮的候选 lexical value 和查询前实际取出的槽位表示上。前者直接训练值抽取器，后者训练真实记忆状态到答案词元的映射。跨轮状态仍然 detach，因此候选级监督也为早期编码器提供了必要梯度。该辅助头仅在训练时提供约束；推理时答案仍由原有自回归 tied LM head 生成，模型没有使用 pointer/copy 机制。

联合训练目标

设答案语言模型损失为 $\mathcal{L}_{\text{LM}}$ ，事实候选的起点、长度和写入监督分别为 $\mathcal{L}_{\text{start}}$ 、 $\mathcal{L}_{\text{len}}$ 和 $\mathcal{L}_{\text{write}}$ ，语义键对齐、读取和保留损失分别为 $\mathcal{L}_{\text{key}}$ 、 $\mathcal{L}_{\text{read}}$ 和 $\mathcal{L}_{\text{ret}}$ 。完整 Value-token Gated 的训练目标为

$$\begin{aligned} \mathcal{L} = & \mathcal{L}_{\text{LM}} + 0.75\mathcal{L}_{\text{start}} + 0.25\mathcal{L}_{\text{len}} + 0.75\mathcal{L}_{\text{write}} + \mathcal{L}_{\text{read}} \\ & + 0.50\mathcal{L}_{\text{key}} + 0.10\mathcal{L}_{\text{ret}} + 0.01\mathcal{L}_{\text{budget}} \\ & + 0.50\mathcal{L}_{\text{v-start}} + 0.25\mathcal{L}_{\text{v-len}} \\ & + 0.50\mathcal{L}_{\text{cand-tok}} + 0.50\mathcal{L}_{\text{retr-tok}}. \end{aligned} \quad (14)$$

其中 $\mathcal{L}_{\text{cand-tok}}$ 在写入轮使用标注值片段， $\mathcal{L}_{\text{retr-tok}}$ 在查询前使用槽位内保存的 lexical value。 $\mathcal{L}_{\text{budget}}$ 惩罚活跃槽位超过目标预算，避免控制器通过无差别激活来

换取答案性能。旧 Gated 只移除四个 value-token 项，SWA-only 不执行事实写入和读取损失；Fixed-LRU 则将写入、保留和淘汰改为确定性规则。这样的定义使各条件的区别集中在长期状态策略，而不是隐藏的训练预算差异。

辅助损失的两个施加位置具有互补作用。候选级损失在事实刚被编码时提供梯度，可以缓解跨轮 detach 造成的信用分配中断；检索级损失使用真实槽位中的 lexical value，检查写入、状态复制和 reader 之后的表示是否仍可解码。若只在候选上监督，模型可能学会从输入事实直接预测值，却没有学会保护和读取槽位；若只在检索后监督，梯度则可能难以回到写入轮的 value encoder。两者同时使用是本次改进的核心训练设计。

可选的稀疏混合专家

为考察计算容量与记忆机制的关系，我们将各解码块的 dense FFN 替换为 4 个结构相同的专家，每个 token 选择概率最高的 2 个专家。若 router 概率为 $p_e(h)$ ，则输出为

$$\text{MoE}(h) = \sum_{e \in \text{Top2}(p(h))} \frac{p_e(h)}{\sum_{j \in \text{Top2}} p_j(h)} F_e(h). \quad (15)$$

当前实现采用 dropless dispatch，所有 token 都会被所选专家处理，并加入负载均衡损失。MoE 只改变 FFN，局部窗口、门控槽位、reader 和值词元对齐路径均保持不变，因此可以检验额外模型容量是否会改变记忆行为或修复词级生成瓶颈。

3 架构改进过程

从局部语言模型到选择性事实记忆

第一阶段先单独训练局部 TinyLM 骨干。在 10M TinyStories 词元上，seed 17 模型的完整验证 NLL 为 2.889852，困惑度为 17.9906。这一阶段关闭事实记忆路径，用于确认固定窗口骨干能够稳定学习；它不构成长程记忆证据。

第二阶段加入结构化事实抽取、写入门、保留门、有限槽位与 Top- K reader。选择性压力测试表明，旧 Gated 模型的目标事实存活率、Recall@1 和 MRR 均达到 100%，平均仅激活一个槽位，且临时事实接收率为 0%。这说明内容门控已经学会保存目标并拒绝噪声。但端到端 EM 只有 52.43%，暴露出检索之后仍存在独立瓶颈。

从失败现象定位词级映射瓶颈

我们首先排除事实丢失和 reader 选错：在全部 288 个样本中，目标槽位均存活且排名第一。随后对表示与目

Table 2: 旧 Gated 模型的关键诊断。检索指标保持不变，性能差异发生在检索后的生成阶段。

诊断条件	EM (%)	Recall@1 (%)
原 combined fusion	52.43	100.00
仅 payload	21.53	100.00
仅 semantic value	27.78	100.00
固定 fusion gate=0.50	57.99	100.00
关闭 fusion gate	5.21	100.00

标词嵌入的余弦相似度进行逐层诊断。payload 平均表示仍携带较强词汇信号，但压缩后的语义 value 与目标词方向并不对齐；经过两层融合后，最终隐藏状态虽恢复了部分目标方向，却不足以稳定压过其他颜色词。正确答案首 token 平均排名为 2.04，有 97.57% 的样本进入 Top-5，而排名第一的比例恰为 52.43%。错误答案通常是另一个颜色词，进一步说明模型学习到了语义类别，但没有形成精确的词元判别边界。

错误的词面分布也呈现明显偏置：137 个错误样本中有 99 个生成了 yellow，其余常见错误为 brown 和 silver；不同目标值的准确率并不均衡，red、gold、yellow 和 amber 达到 100%，而部分其他颜色在该 run 中几乎无法排到第一。这种结果与“模型知道答案属于颜色集合，但受词频和 tied embedding 几何影响而偏向某个竞争词”的解释相符，也说明只看平均 EM 还不足以描述词汇恢复的难度。

推理干预也支持这一判断。仅使用 payload 或仅使用 semantic value 都比二者结合更差，说明两条旧表示都含有部分有效信息；把融合门固定为 0.5 可将 EM 从 52.43% 小幅提高到 57.99%，但无法消除大部分错误。旧模型的 EM 从 step 500 的 11.46% 持续升至 step 2000 的 52.43%，表明延长训练可能继续改善生成，却没有为记忆表示建立直接的词表监督。

加入值片段和直接词元监督

根据上述证据，第三阶段不再复杂化写入或读取控制器，而是补充值片段抽取、lexical value 状态和 memory-to-token 辅助损失。改动后，旧模型已经取得的目标存活率、Recall 和槽位占用均保持不变，端到端 EM 则从 52.43% 提升到 100%。正确答案首 token 的平均排名变为 1.00，最终 logit margin 达到 +13.5763。这构成一条一致的因果证据链：先观察到“检索正确而生成失败”，再确认正确词通常已接近词表顶部，最后用直接的值词元对齐修复排名不稳定性。

MoE 扩展作为机制检验

最后，我们在四种记忆条件上分别加入相同的 4-expert、Top-2 MoE。该扩展的目的既是验证稀疏 FFN 与有界记忆能否共存，也是检验增加参数容量能否自然解决旧模型的生成问题。结果显示，旧 Gated 加入 MoE 后仍保持满分检索，但 EM 从 52.43% 降至 32.99%；完整值词元模型则在 Dense 和 MoE 下均为 100%。因此，MoE 提供的条件计算容量并不能替代明确的 memory-to-token 学习目标。

4 实验设置

模型与优化配置

所有正式实验从同一个 seed-17 TinyStories 骨干初始化。TinyLM 包含 6 层、384 维隐藏状态、8 个注意力头和 1536 维 FFN，词表大小为 50,257。局部 KV 预算为 128，长期用户记忆为 8 个槽位，读取 Top-K 为 4，每轮最多提出一个写入候选。合并相似度阈值为 0.999；训练与评估时写入/保留阈值分别为 0.1/0.1 和 0.5/0.5。

模型使用 AdamW， $\beta = (0.9, 0.95)$ ，weight decay 为 0.1。骨干学习率为 3×10^{-5} ，记忆控制器和新增 head 的学习率为 3×10^{-4} 。每个条件训练 2000 个优化步，physical batch 为 2，梯度累积为 2，有效 batch 为 4；梯度范数裁剪为 1.0。计算采用 RTX 4090 上的 BF16 autocast。值对齐模型的 value-start、value-length、候选 memory-token 与检索后 memory-token 损失权重依次为 0.50、0.25、0.50 和 0.50。

MoE 条件使用 4 个专家、Top-2 droppless routing 和权重为 0.01 的负载均衡项。所有 Dense/MoE 条件共享骨干初始化、训练预算、数据分布和评估协议。

每个条件在正式训练前均执行单元测试和短步数冒烟运行，确认状态重排、reset、值片段 mask、辅助损失梯度以及 MoE dispatch 形状正确。正式运行每隔 500 step 保存 checkpoint，并在最终 checkpoint 上完成全量评估；因此可以同时复核最终质量和训练后期的稳定性。

选择性记忆压力测试

每条样本按以下顺序构造：一条需要长期保存的 durable user fact，若干与其形式相同但标为临时的 user facts，一段 TinyStories 自然文本延迟，以及针对 durable fact 的查询与答案。事实关系固定为“人物喜欢某个值”，值集合含 16 个颜色词。训练使用 Remember/Keep/Save 作为长期提示，使用 Temporary/Skip/Ignore 作为临时提示；验证改用未见的 Store 和 Discard，以减少对单个提示词的直接记忆。

训练噪声数为 0/1/2/4，延迟为 1/2/4 个长度为 128 的 segment。验证采用 0/2/4/8 条临时事实与 1/4/16 个延迟 segment 的笛卡尔积，共 12 个条件；每个条件 24 条样本，总计 288 条。最长延迟约为 2048 个 TinyStories 干扰词元，远超 124 个 recent KV。验证中的最大噪声数等于长期槽位总数，且在 Fixed-LRU 中会与 durable fact 竞争容量。

对比条件与评价指标

实验比较四种记忆策略及其 MoE 版本：(1) SWA-only，仅有 sink 与 recent KV；(2) Fixed-LRU，所有有效候选均写入，容量满后按 LRU 淘汰；(3) old Gated，使用自适应写入和保留，但没有值词元对齐；(4) Value-token Gated，在旧门控模型上加入完整的值片段与词元监督路径。

端到端质量以 greedy generation 的 Exact Match (EM) 和 teacher-forced answer NLL 衡量。记忆行为使用目标事实存活率、Recall@K、MRR、平均活跃槽位数与噪声接收率。值路径还记录 value 起点、长度、片段 exact，以及 memory-token 的 token accuracy 和 sequence exact。SWA-only 不包含长期槽位，因此其 survival 和 recall 记为 N/A，而不是零。

本文对这些指标作如下定义。若目标槽位在回答前仍为 active, 则计为一次 survival 成功; 若其在 reader 排名中位于前 K , 则计为 Recall@ K 成功。对单目标任务, 目标排名为 r 时, 倒数排名为 $1/r$, 所有样本的平均值构成 MRR。EM 要求完整生成序列 (包括结束符之前的答案词元) 与标准序列完全相同; 因此, 模型生成另一个颜色词、漏掉一个多词元片段或在答案后继续生成都会计为错误。noise accept rate 是被硬写入的临时事实候选数除以临时候选总数, 反映控制器是否把容量让给不应长期保存的信息。

teacher-forced NLL 只在标准答案位置计算, 用于区分“模型已将正确词放在高概率位置”和“自由生成真正完成答案”这两个层面。value-span exact 要求起点和长度同时正确; memory-token sequence exact 要求 lexical value 预测的全部有效位置逐一正确。对于多词元值, 这些指标比首 token accuracy 更严格, 也能发现模型只预测了值的前缀却没有正确终止的情况。

5 实验结果

Dense 与 MoE 的总体比较

表 3 汇总八个正式条件。Dense Value-token Gated 在 288/288 条验证样本上达到 100% EM、目标存活率和 Recall@1, 平均只使用一个槽位, 并拒绝全部临时事实。它相对 old Gated 提高 47.57 个百分点 EM, 而检索和槽位占用没有变化; 相对 Fixed-LRU 提高 25.35 个百分点 EM, 并少使用 3.25 个槽位。

Fixed-LRU 的 Dense EM 为 74.65%, 高于 old Gated 的 52.43%, 但它接收了 75% 的临时事实, 平均使用 4.25 个槽位, 目标存活率降至 92.01%。这说明较高 EM 并不等价于更可靠的选择性记忆: 它在部分样本中能利用保存的词元信息生成答案, 却会在容量竞争中丢失早期目标。门控模型以更少槽位取得满分 survival 和 recall, 值对齐路径再将这一可靠检索转化为精确生成。

加入 MoE 后, SWA-only 的 EM 仍为 5.21%; Fixed-LRU 下降 9.38 个百分点, old Gated 下降 19.44 个百分点, 而 Value-token Gated 保持 100%。MoE 没有改变后两种门控模型的 survival、Recall 和 MRR。该结果表明, 路由到更多 FFN 参数既不会自动产生长期状态, 也不会自动解决连续记忆到离散答案的映射; 明确的记忆结构和训练目标仍是决定性因素。

长延迟、高噪声与词级恢复

Value-token Gated 在全部 12 个验证条件中均取得 100% EM, 包括 8 条临时事实和 16 个延迟 segment 的最难组合。此时目标事实仍以 100% 概率存活并排名第一, 平均活跃槽位保持为 1.00。因此, 在当前压力范围内没有观察到随延迟或噪声增强而退化。

值路径的各项指标也达到满分: value start accuracy、value length accuracy、value span exact、memory-token token accuracy 和 sequence exact 均为 100%。正确答案首 token 的平均排名为 1.0000, Top-5 为 100%, 相对次高 logit 的平均 margin 为 $+13.5763$, teacher-forced answer NLL 为 1.95×10^{-5} 。所有生成都以正确值序列开头并随后输出 EOS; 其中包含被 GPT-2 tokenizer 拆为两个 token 的 navy, 说明评测并未只覆盖单 token 值。

为了检查结果是否只出现在某个容易的网格单元, 我们进一步按噪声数量和延迟长度聚合。Dense Value-

token Gated 在 0/2/4/8 条临时事实和 1/4/16 个延迟 segment 的每个组合中均为 100% EM、100% survival 和 100% Recall@1。对照的 Dense Fixed-LRU 在噪声数为 8 时目标 survival 为 68.06%、Recall@1 为 61.11%, 平均使用全部 8 个槽位; Dense old Gated 仍保持满分 survival 和 Recall, 但其 EM 在四个噪声桶中分别为 48.61%、51.39%、54.17% 和 55.56%。这组结果将“是否保留目标”和“能否正确作答”清楚分开: old Gated 的检索已经稳定, 而 Value-token Gated 进一步修复了词级输出。

按延迟聚合时, Dense old Gated 在 1/4/16 个 segment 上的 EM 分别为 47.92%、58.33% 和 51.04%, 但三个延迟下的 survival/Recall@1 都为 100%。由于不同网格单元使用不同确定性样本索引, 这些 EM 波动不能被解释为严格的单调长度曲线; 能够稳健外推的是长期槽位本身的存活和读取, 而不是某个固定事实在不同 filler 长度下的逐样本差值。

门控输出的只读审计进一步揭示了控制器的行为。288 个 durable 候选的写入概率最小、平均和最大值均为 1.000000 ; 1,008 个 temporary 候选的写入概率平均为 5.69×10^{-7} , 最大值为 2.62×10^{-5} , 全部在硬阈值下被拒绝。目标槽位的保留概率均值为 0.999239, 范围为 $[0.999117, 0.999296]$ 。两个融合层的查询门均值分别为 0.999295 和 0.984094, 说明旧 Gated 的低 EM 不能主要归因于记忆被门控关闭。不过, 这些概率在当前有明确 durable/temporary 提示的任务上接近饱和; 对于重要性模糊或事实过期的真实对话, 是否能够主动降低保留权重仍需测试。

训练进程与 checkpoint 稳定性

表 5 显示, 完整模型在 step 500 已达到 100% EM; 之后训练主要提高输出置信度, 并使辅助头对多 token 值完全收敛。首 token margin 从 $+6.8794$ 持续增至 $+13.5763$, teacher-forced NLL 同步下降。四个 checkpoint 均为满分 EM, 也降低了结果只由最终单点偶然出现的可能性, 但不能替代跨随机种子实验。

参数、状态与运行开销

表 6 给出主要资源指标。值片段与词元路径使 Dense 模型参数增加 305,293, 相当于 0.96%; 完整 recurrent state 增加 14,796 bytes, 即 0.61%。单个活跃槽位的逻辑字节数因新增 384 维 lexical vector 与值 ID/mask 增加 51.0%, 但局部六层 KV cache 占据了完整状态的主要部分, 因此总体增幅较小。训练时间增加 4.31%, 峰值 allocated 显存增加 0.36%。

MoE 将 Value-token 模型总参数从 32.16M 增加到 60.48M, 而每 token 激活约 39.25M 参数。与对应 Dense 模型相比, 其训练时间增加 39.6%、吞吐下降 28.4%、峰值 allocated 显存增加约 19.5%。该开销反映普通 PyTorch token indexing、逐专家前向和 index_add 原型, 不能代表 fused grouped-GEMM 等优化实现的上限。

最终 checkpoint 的路由审计未显示 expert collapse。Value-token Gated + MoE 的四个专家 token fraction 为 0.471/0.508/0.511/0.511, 加权 dispatch fraction 为 0.239/0.259/0.244/0.258, 路由熵为 1.355。在 Top-2 设置下, 均匀 token fraction 的参考值约为 0.5, dispatch fraction 的参考值约为 0.25; 因此四个专家均获得了有效负载。

Table 3: Dense 与 MoE 条件的总体结果。Surv. 表示目标事实存活率；Noise acc. 表示临时事实接收率。所有数值均为 seed 17 的最终 checkpoint 点估计。

FFN	记忆策略	EM (%)	Answer NLL	Surv. (%)	R@1 (%)	MRR	Slots	Noise acc. (%)
Dense	SWA-only	5.21	1.38947	N/A	N/A	N/A	0.00	N/A
	Fixed-LRU	74.65	0.50323	92.01	90.28	0.9103	4.25	75.00
	old Gated	52.43	0.63089	100.00	100.00	1.0000	1.00	0.00
	Value-token Gated	100.00	0.00001946	100.00	100.00	1.0000	1.00	0.00
MoE	SWA-only	5.21	1.38736	N/A	N/A	N/A	0.00	N/A
	Fixed-LRU	65.28	0.57731	93.06	93.06	0.9306	4.25	75.00
	old Gated	32.99	0.85217	100.00	100.00	1.0000	1.00	0.00
	Value-token Gated	100.00	0.00002212	100.00	100.00	1.0000	1.00	0.00

Table 4: Dense 条件按临时事实数量聚合的结果。每行包含三个延迟条件，共 72 条样本。

Noise	old EM (%)	old Surv. (%)	old R@1 (%)	Fixed EM (%)	V3 EM (%)
0	48.61	100.00	100.00	73.61	100.00
2	51.39	100.00	100.00	83.33	100.00
4	54.17	100.00	100.00	80.56	100.00
8	55.56	100.00	100.00	61.11	100.00

Table 5: Value-token Gated 的 checkpoint 回扫。Aux. exact 为辅助值序列完全正确率。

Step	EM (%)	Rank	Margin	Aux. exact (%)
500	100	1.00	6.8794	94.79
1000	100	1.00	9.1122	100.00
1500	100	1.00	11.4877	100.00
2000	100	1.00	13.5763	100.00

与六种基础架构及滑动窗口参考的综合比较

为避免将不同实验协议下的数值误读为同一排行榜，本节分三层报告结果。第一层是统一 TinyStories/TinyLM 的十百万预测词元实验，用于比较 Full Attention、Longformer、Memformer、Linformer、Performer 和 Reformer 六种训练型路径 (Beltagy et al. 2020; Wu et al. 2020; Wang et al. 2020; Choromanski et al. 2021; Kitaev et al. 2020)；第二层是同一骨干上的 Keyformer 推理期缓存压缩；第三层是本文的选择性事实记忆压力测试。前两层主要衡量语言模型困惑度与系统开销，第三层则衡量跨轮事实的写入、保留、检索和精确回答。因此，表中的“N/A”表示该架构在相应协议下没有定义该指标，而不是性能为零。

六种训练型架构的 Dense 比较。 表 8 是本节的跨双栏总表，汇总统一 TinyStories 协议下的 Dense FFN 结果。训练吞吐包含周期性 probe 和 checkpoint；为便于直观比较，训练时间由 10^7 个预测词元除以该吞吐得到，是 run-level 墙钟时间的近似值。Full Attention 和 Reformer 的结果来自 C v2 协议，其他四种结果来自 A+B unified 协议；它们共享 TinyLM 维度和主要优化设置，但仍存在 runner 细节差异，故表中结论应理解为机制级而非严格等条件因果比较。

在该统一筛选中，Memformer 的 PPL 最低，但它比公共主干多约 11.83% 参数；Longformer 的质量接近

Table 6: 架构扩展的资源开销。Active/token 是每个 token 参与计算的参数量；当前状态张量按最大槽位预分配，逻辑活跃字节不等于物理显存节省。

模型	总参数	Active/token	State bytes	训练时间 (s)	吞吐 (ex/s)	Peak alloc. (GB)
old Gated Dense	31.85M	31.85M	2,408,906	1323.0	6.047	2.673
Value-token Dense	32.16M	32.16M	2,423,702	1380.1	5.797	2.682
old Gated MoE	60.18M	38.94M	2,423,702	1905.0	4.20	3.201
Value-token MoE	60.48M	39.25M	2,423,702	1927.1	4.15	3.205

Table 7: MoE 路由与负载均衡审计。Token fraction 的四项之和约为 2，dispatch fraction 的四项之和约为 1。

条件	Token fraction (E0/E1/E2/E3)	Dispatch fraction	熵
SWA-only	.536/.434/.574/.456	.251/.242/.273/.233	1.230
Fixed-LRU	.532/.527/.533/.408	.297/.238/.262/.202	1.282
old Gated	.545/.457/.569/.429	.275/.232/.270/.223	1.312
Value-token	.471/.508/.511/.511	.239/.259/.244/.258	1.355

Memformer，却承担了更高的峰值显存；Linformer 和 Performer 具有较高训练吞吐，但当前低秩或随机特征近似会带来质量损失。Full Attention 的吞吐最高主要得益于成熟的 SDPA 路径，不能简单将这一工程优势等同于更低的理论复杂度。Reformer 的吞吐接近 Full Attention，但其质量低于前述三种路径，并且 LSH 候选构造的因果性和稀疏 kernel 实现更加敏感。上述结果说明，减少理论 attention 复杂度并不自动转化为端到端速度优势；训练吞吐还受到张量布局、Python 调度、排序和投影开销的影响。

MoE 对六种骨干的影响。 在全部六层 FFN 中加入四专家 Top-1 dropless MoE 后，统一 seed-17 screening 结果如表 9 所示。MoE 增加了总参数容量，但每个 token 只选择一个专家，因此激活参数仍接近 Dense FFN。表中 MoE 时间同样按十百万预测词元的工作流吞吐换算，不能与专用 fused grouped-GEMM 的理论峰值混同。

相对于同 seed 的 Dense 版本，MoE 对 Memformer 的 PPL 改善最大 (约 8.30%)，对 Longformer 和 Linformer 只有约 1.43% 和 0.84%，对 Performer 几乎没有改善。这一现象不应被解释为 MoE 普遍增强了记忆能力：Memformer 同时保留了额外的 recurrent memory 参数，且所有 MoE 结果只有一个 seed。更稳妥的解释是，条件 FFN 容量与不同 attention 状态路径存在交互，但 attention 近似误差或长期信息缺失不能仅靠扩大 FFN 容量解决。当前普通 PyTorch dispatch 使训练吞吐下降约 7%–27%、峰值显存增加约 6%–15%；这反

Table 8: 总表：六种基础架构的 Dense FFN 结果。时间为十百万预测词元的近似墙钟时间；Keyformer 不属于训练型架构，故单列于后文。

架构	主要状态/压缩路径	Val PPL ↓	吞吐 (tok/s) ↑	约训练时间 (min) ↓	Peak alloc.	参数量	长期事实路径
Full Attention	完整 causal KV	18.95 ± 0.12	53,770 ± 585	3.10	2.58 GiB	29.93M	无
Longformer	固定左侧窗口 $W = 128$	17.91 ± 0.08	13,499 ± 269	12.35	6.80 GiB	29.93M	无
Memformer	64 个 recurrent slots	17.27 ± 0.08	14,670 ± 301	11.36	2.63 GiB	33.47M	固定摘要
Linformer	K/V 序列低秩投影	18.80 ± 0.11	27,858 ± 106	5.98	2.58 GiB	29.93M	无
Performer	FAVOR+ 随机特征	31.52 ± 0.26	27,079 ± 541	6.15	2.70 GiB	29.93M	无
Reformer	因果 LSH 候选	20.58 ± 0.16	46,939 ± 954	3.55	2.57 GiB	29.04M	无

Table 9: 总表：六种骨干在全层 MoE 下的 seed-17 结果。Keyformer 使用 Full-Attention+MoE 检查点，但本身不参与训练。

架构	Val PPL ↓	吞吐 (tok/s) ↑	约训练时间 (min) ↓	Peak alloc.	总参数	激活参数/token
Memformer	15.7831	12,201	13.66	2.96 GiB	54.71M	33.48M
Longformer	17.6290	12,265	13.59	7.23 GiB	51.17M	29.93M
Full Attention	18.3174	26,504	6.29	2.91 GiB	51.17M	29.93M
Linformer	18.6405	20,340	8.20	2.91 GiB	51.17M	29.93M
Reformer	19.7071	26,821	6.22	2.90 GiB	50.28M	29.05M
Performer	31.7477	20,359	8.19	3.12 GiB	51.17M	29.93M

映原型实现而非 fused MoE kernel 的上限。

Keyformer 的推理期比较。 Keyformer 改变的是已训练模型的 KV-cache，而不是训练网络。Dense Full-Attention 检查点上，FullKV、Keyformer-75 和 Keyformer-50 的 PPL 分别为 14.91、14.91 和 14.89，平均 KV 字节数从 4,709,376 降至 3,538,944 和 2,359,296。在当前实现中，Keyformer-50 的 decode 吞吐由 219.8 降至 184.2 token/s，下降约 16.2%。在 Full-Attention+MoE 检查点上也观察到一致方向：KV 字节减少 50%，PPL 基本不变，但吞吐下降约 17.2%。因此，Keyformer 的主要收益是显存预算，而非当前 Python 选择与 gather 路径下的速度；它不应与六种训练型架构共享一个训练时间排名。

所给滑动窗口文献的 SWA-sink 参考。 用户指定的“Sliding-window beats linear attention”文献（作者来自 Microsoft Applied Sciences Group 与 Independent，并非 Google 官方论文）将四个 attention sink 与固定滑动窗口结合，报告了无需后训练的推理路径。在其多模型下游恢复率实验中，SWA(窗口 64、sink 4) 的 MMLU 恢复率为 $93.2 \pm 3.5\%$ ，六项任务平均恢复率为 $99.0 \pm 0.5\%$ ；作为对照，LoLCATs、Liger-GLA 和 QLinAtt 的平均恢复率分别为 $97.5 \pm 1.3\%$ 、 $92.0 \pm 2.8\%$ 和 $92.9 \pm 0.0\%$ (Jolicoeur-Martineau et al. 2026)。该文的 SWA 结果没有新增训练 token 或后训练阶段，因此训练改造开销为零，但它仍然只保留局部 token KV，不能保证跨越窗口的任意事实可被逐字恢复。本文采用的 SWA-sink 局部通道与该参考共享“固定窗口 + 少量 sink”的思想，但额外维护内容驱动的事实槽位；两者在状态语义上并非同一模型。

需要区分论文来源与实现来源：Google Gemma-PyTorch 中的 LOCAL_SLIDING 是局部注意力的官方实现参考，而“Sliding-window beats linear attention”本身的作者署名并非 Google，且截至本实验记录日期没有可确认的作者代码仓库。因此，本文将该 PDF 的结论作

Table 10: 栏目分析表：本文事实记忆与六种基础路径的机制级对照。该表只服务于机制分析，采用单栏排版；六种基础架构的长期事实指标在当前 TinyStories 语言建模协议中未测量，故不以 N/A 伪装成零。

方案	训练时是否改变 attention/状态	长期状态形式	是否能内容级抑制噪声	主要记忆风险
Full Attention	是，完整 attention	全历史 KV，随长度增长	否	KV 无界增长
Longformer	是，固定窗口	sink+recent token KV	否	窗口外事实不可直接访问
Memformer	是，segment recurrent state	固定数量摘要槽	否（当前基线）	压缩造成细节衰减
Linformer	是，低秩 K/V	投影后的序列表示	否	高秩局部信息损失
Performer	是，随机特征 kernel	前缀统计量/近似状态	否	近似误差与覆盖问题
Reformer	是，LSH 候选	哈希桶连接	否	碰撞删除，候选不完整
Keyformer	否，仅保留 cache	重显旧 KV+recent KV	部分（按重要性）	选择错误后历史不可恢复
本文 Value-token Gated	是，局部 attention 外接事实记忆	8 个内容门控语义/词汇槽	是，写入 + 保留 + 合并	依赖事实边界与门控泛化

为外部 SWA-sink 证据，将 Gemma 的 LOCAL_SLIDING 仅作为代码级窗口构造参考；二者都不与本文 TinyLM 的绝对 PPL、训练时间或事实记忆 EM 直接合并。

Google Gemma-PyTorch 中的 LOCAL_SLIDING 作为局部滑动窗口的代码参考，但并未在上述论文表中作为一个独立的 benchmark 方法报告，因此其对应数值记为未测量。本文的 4 个 sink 加 124 个 recent token 与该类局部窗口设计具有相同的状态约束思想，但额外添加了内容门控事实槽位。

事实记忆压力测试中的端到端对照。 六种基础架构的 TinyStories PPL 不能回答“重要事实数千个干扰词元后是否仍被保留”。因此，本文进一步在独立的选择性记忆协议中报告记忆指标。该协议包含 288 条验证样本、最多 8 条临时事实和 16 个延迟 segment，训练预算为 2000 步；它与上文十百万预测词元的语言建模实验不共享绝对时间单位。Dense 与 Top-2 MoE 的关键结果为：

该完整清单显示，Value-token Gated 相比 old Gated 只增加约 0.96% Dense 参数和 4.31% 训练时间，却将 EM 从 52.43% 提升至 100%。相同记忆路径切换到 Top-2 MoE 后，总参数增加约 88.9%，训练时间增加 39.6%，吞吐下降 28.4%，峰值 allocated 显存增加约 19.5%；因此“稀疏激活”不等于“低总存储或低训练成本”。

Table 11: 外部滑动窗口与线性注意力文献的结果清单。数值来自 (Jolicoeur-Martineau et al. 2026) 的恢复率表，不能与本文 TinyLM 的 PPL、EM 或训练时间直接合并；“—”表示原文未报告。

方法	后训练词元	阶段数	MMLU 恢复率 (%)	六任务平均恢复率 (%)
SUPRA	100B	1	53.0±0.0	88.1±0.0
Hedgehog	40M	2	36.9±0.0	73.9±0.0
LoLCATs	40M	2	83.2±2.2	97.5±1.3
Liger-GLA	20M	1	62.2±5.8	92.0±2.8
MOHAWK	3–5B	3	56.9±0.0	92.4±0.0
Mamba in the Llama	20B	2	67.7±0.0	86.7±0.0
DiJiang	40B	1	88.7±0.0	—
ARWKV	60M/830M	2/3	84.1±0.0	94.7±0.0
Llama	8–12B	3	91.5±0.0	98.6±0.0
QLinAtt	350–700M	3	74.0±0.0	92.9±0.0
QRWKV6	350–700M	3	92.4±2.7	99.1±0.8
QRWKV7	350–700M	3	86.4±6.8	96.1±4.1
SWA($w = 64, s = 4$)	0	0	93.2±3.5	99.0±0.5

Table 12: 选择性事实记忆压力测试。这里的训练时间是 2000 步 run 的实际墙钟时间，不能与表 8 的十百万词元时直接相减。

FFN	记忆条件	EM ↑	Answer NLL ↓	Survival ↑	Recall@1 ↑	Avg. slots ↓	Noise accept ↓	训练时间 (s)
Dense	SWA-only	5.21%	1.38947	N/A	N/A	0.00	N/A	880.1
Dense	Fixed-LRU	74.65%	0.50323	92.01%	90.28%	4.25	75.00%	1200.6
Dense	old Gated	52.43%	0.63089	100.00%	100.00%	1.00	0.00%	1323.0
Dense	Value-token Gated	100.00%	0.00001946	100.00%	100.00%	1.00	0.00%	1380.1
MoE	SWA-only	5.21%	1.38736	N/A	N/A	0.00	N/A	1479.8
MoE	Fixed-LRU	65.28%	0.57731	93.06%	93.06%	4.25	75.00%	1804.0
MoE	old Gated	32.99%	0.85217	100.00%	100.00%	1.00	0.00%	1905.0
MoE	Value-token Gated	100.00%	0.00002212	100.00%	100.00%	1.00	0.00%	1927.1

该表揭示了本文架构相对于六种基础 attention 路径的额外能力。SWA-only 与一般固定窗口路径一样，不能在局部 KV 之外提供稳定的事实恢复；Fixed-LRU 虽然获得较高 EM，却接收 75% 的临时事实，且平均占用 4.25 个槽位。old Gated 已经解决选择性存储问题，但在没有值词元对齐时，EM 仍只有 52.43%（加入 MoE 后为 32.99%）。Value-token Gated 在 Dense 与 MoE 下均达到 100% EM、100% survival 和 Recall@1，同时只激活一个事实槽位并拒绝全部临时事实。这表明本文的主要新增贡献不是单纯扩大 FFN 或局部窗口，而是把长期记忆管理和词级解码路径显式拆开。

从训练开销看，Value-token 路径相对于 old Gated Dense 增加 4.31% 的训练时间、0.36% 的峰值 allocated 显存和 0.61% 的完整 recurrent state；相对于对应 Dense 模型，Top-2 MoE 使 Value-token 条件的训练时间增加 39.6%、吞吐下降 28.4%、峰值显存增加约 19.5%，总参数量由 32.16M 增至 60.48M。换言之，本文架构以很小的 Dense 记忆扩展换取了显著的词级恢复收益，而 MoE 的主要作用是增加条件 FFN 容量，其资源成本不能被“每 token 只激活少数专家”这一事实完全抵消。

6 结果分析

选择性记忆的收益来自容量分配

SWA-only 的结果说明固定局部窗口无法可靠恢复已经离开 recent KV 的事实。Fixed-LRU 证明了保存更多事实确实能改善部分样本的回答，但它把临时事实和持久

事实同等处理，因此在噪声增长时发生目标淘汰。Gated 模型的平均槽位数为 1.00、噪声接收率为 0%，同时保持 100% survival，说明其收益来自对有限容量的内容相关分配，而不是扩大状态或无差别写入。

这一结果也给出了门控指标与端到端指标的不同含义。Survival 和 Recall 回答“正确事实是否仍在、reader 是否找到了它”；EM 回答“生成器是否把它转成了正确字符串”。若只报告 EM，Fixed-LRU 可能显得优于旧 Gated；若同时观察状态与检索，则可发现旧 Gated 已经完成了更可靠的选择性存储，只是尚未解决词级输出。因而，对外部记忆模型的评价应覆盖写、留、读、答四个环节。

语义检索与精确生成之间存在表示鸿沟

old Gated 的 Recall@1 为 100% 而 EM 为 52.43%，是本实验最关键的诊断现象。完整事实平均池化将命令词、人物、关系、值与标点混合到一个向量中；payload embedding 的平均又丢失了词元顺序和值边界。这样的连续表示足以支持语义相似度检索，却没有被要求与词表中的某个离散 token 对齐。tied LM head 进一步要求最终隐藏状态落在正确 token embedding 的方向上，轻微的表达偏差便可能让另一个颜色词取得最高 logit。

值片段建模通过职责分离缓解了这一矛盾：semantic key/value 保存完整关系并服务检索，lexical value 聚合目标子片段并服务生成。memory-to-token CE 则把原先只经最终 LM loss 间接学习的路径改为可直接优化的映射。模型没有改变槽位数、reader 排名或噪声过滤，

Table 13: 选择性记忆实验的完整训练与存储开销清单。时间、吞吐和显存来自 2,000 步 seed-17 run; state bytes 是完整 recurrent state 的逻辑大小, 不能等同于 GPU 物理显存。

FFN	条件	总参数	Active/token	时间 (s)	吞吐 (ex/s)	Peak alloc. (GB)	Peak reserved (GB)	State bytes	Active slots
Dense	SWA-only	31.85M	31.85M	880.1	9.09	2.871	6.881	2,423,702	0.00
Dense	Fixed-LRU	31.85M	31.85M	1200.6	6.66	2.666	6.568	2,423,702	4.25
Dense	old Gated	31.85M	31.85M	1323.0	6.05	2.673	6.648	2,423,702	1.00
Dense	Value-token Gated	32.16M	32.16M	1380.1	5.80	2.682	6.702	2,423,702	1.00
MoE	SWA-only	60.18M	38.94M	1479.8	5.41	3.433	7.176	2,423,702	0.00
MoE	Fixed-LRU	60.18M	38.94M	1804.0	4.43	3.190	6.960	2,423,702	4.25
MoE	old Gated	60.18M	38.94M	1905.0	4.20	3.201	7.021	2,423,702	1.00
MoE	Value-token Gated	60.48M	39.25M	1927.1	4.15	3.205	6.984	2,423,702	1.00

却获得 47.57 个百分点的 EM 提升, 这与“词级对齐是主要瓶颈”的假设高度一致。

不过, 当前实验同时加入了 value span、lexical fusion 和 token auxiliary loss, 尚不能单独归因每一项的贡献。满分结果表明这组组合在受控任务上充分有效, 但不能证明这是唯一或最小的解决方案。token-level cross-attention、显式 pointer/copy 或 untied output head 仍是值得比较的替代路径。

MoE 与外部记忆解决不同问题

MoE 增加每层 FFN 的条件计算容量, 外部事实槽位则扩展模型跨轮保存信息的时间尺度。SWA-only 加入 MoE 后仍处于偶然命中水平, 说明参数容量无法重建已经从局部状态中删除的事实。old Gated + MoE 仍能满分检索却更难精确作答, 说明专家容量也不会自然形成从记忆向量到词表的监督。只有显式值对齐模型在两种 FFN 下均保持 100% EM。

因此, 当前证据支持二者的模块化关系: 门控事实记忆负责选择和长期保存, 值路径负责精确解码, MoE 提供可选的条件计算容量。MoE 没有破坏完整模型的记忆行为, 也未发生明显路由坍塌, 但在当前任务已被 Dense 模型解决的情况下, 它没有带来可见质量收益, 反而增加了训练时间、显存和总参数存储。

7 不足之处

首先, 全部正式结果仅来自 seed 17。表中的 100% 是单次运行在 288 条验证样本上的点估计, 尚不能报告均值、标准差、置信区间或统计显著性。checkpoint 回扫显示同一训练轨迹较稳定, 但它不能衡量不同初始化和数据顺序带来的变化。

其次, 压力测试仍是封闭的合成任务。答案集合只有 16 个颜色值, 关系固定为“人物喜欢颜色”, 提示模板数量有限。尽管验证使用了未见的 durable/noise cue、训练范围外的延迟和更高噪声, 模型仍可能学习颜色分类与固定句法映射。当前结果不能外推为开放域对话中的实体、数字、日期、代码、任意字符串或知识更新能力。

第三, 值边界来自数据生成器提供的人工监督。真实对话没有天然的 value offset, 模型需要依靠自动标注、弱监督、关系抽取或联合学习来发现应被精确恢复的子片段。现有辅助头也不是 pointer/copy head; 面对 OOV 词元、很长的数字串或罕见多 token 实体, 普通 tied LM head 仍可能失败。

第四, 完整新模型缺少内部消融。value-span 监督、lexical state、semantic-lexical fusion、候选级 token loss 和检索后 token loss 同时启用, 实验只能评价它们的组

合效果。旧 Dense run 的 state schema 与后续实现略有差异, 统一表中的旧模型状态字节由当前 schema 只读复算; 这一处理便于配对, 但不代表历史 checkpoint 的物理布局发生变化。

第五, 当前实现仍是研究原型。槽位张量按最大容量预分配, 较少的逻辑活跃槽位不会直接减少物理显存; 滑动注意力和 MoE dispatch 也没有使用生产级 fused kernel。推理只测试 greedy argmax, 尚未比较采样、beam search 或更开放的自由生成。训练和评估均使用最终 checkpoint, 没有在独立 validation set 上选择最佳停止点。

8 未来展望

近期工作首先应补充 seed 29 和 43, 在完全相同的训练和验证协议下报告三种子均值与标准差。随后应对词级路径做最小消融: 分别移除 token loss、保持旧 fusion、只启用候选级监督或检索后监督, 以区分显式边界、词汇状态与直接解码目标的独立贡献。对于门控机制, 还应扫描槽位容量、Top-K、写入与保留阈值, 并采用弱化的重要性提示, 确认控制器学习的是内容特征而不是少数 cue 的词面规则。

任务层面需要扩展值类型与语言表达, 包括训练中未出现的人名、日期、整数、小数、长数字、罕见多 token 字符串和随机标识符; 同时增加多关系事实、查询改写、否定、冲突更新与同一实体的多属性。容量压力应超过八个槽位, 并测试事实在多轮更新后的版本一致性。只有当模型在这些条件下仍能保持高 survival 和 recall, 却出现精确输出退化时, 才有充分依据引入显式 pointer/copy 或 token-level cross-attention, 并量化其准确率、状态和延迟代价。

工程层面可实现物理压缩的活跃槽位与 fused sliding-window attention, 避免将逻辑状态节省误解为显存节省。MoE 则应在 fused grouped-GEMM 或 MegaBlocks 类 dispatch 下重新测量吞吐, 并在更复杂任务上判断其额外容量是否带来收益。未来更理想的系统应形成清晰的分层: 局部 KV 处理逐词精确上下文, 语义槽位负责跨轮选择性保持, 词元级路径恢复需要原样输出的值, MoE 仅在任务复杂度确实需要时提供额外条件容量。

参考文献

References

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150.

Alexia Jolicoeur-Martineau, Rhea Sanjay Sukthanker, Pashmina Cameron, and Emy Gervais. 2026. Sliding-window beats linear attention. arXiv preprint arXiv:2608.28444.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Advances in Neural Information Processing Systems.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In International Conference on Learning Representations.

Qingyu Wu, Zhenzhong Lan, Jingyu Li, and others. 2020. Memformer: A memory-augmented transformer for sequence modeling. arXiv preprint arXiv:2010.06891.

Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. arXiv preprint arXiv:2006.04768.

Krzysztof Choromanski, Valerii Likhoshesterov, Dohan Yoon, et al. 2021. Rethinking attention with performers. In International Conference on Learning Representations.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. In International Conference on Learning Representations.